

2.1 Ejercicio 2.1. Tipos primitivos

- a) **La edad de una persona.** Numéricamente cabría en un byte (rango -128 a 127), pero en la práctica se usa int: las operaciones aritméticas con byte se promueven automáticamente a int en Java, así que el ahorro de memoria no suele compensar la incomodidad de tener que castear continuamente.
- b) **El saldo de una cuenta bancaria.** Requiere parte decimal, por lo que el tipo adecuado es double. (En una aplicación real de gestión financiera se recomendaría BigDecimal en lugar de un tipo primitivo, precisamente para evitar los errores de redondeo propios de la representación en coma flotante, pero entre los tipos primitivos double es la opción correcta.)
- c) **La inicial del nombre de un usuario.** Un único carácter se representa con char.
- d) **Si un usuario está activo o no.** Es un valor de verdadero/falso, por tanto boolean.
- e) **La distancia en metros entre la Tierra y la Luna.** Unos 384 400 000 m, que cabe perfectamente en el rango de int (hasta ~2 147 millones) si se expresa como número entero de metros. Si se necesitara precisión decimal (fracciones de metro), habría que usar double.

2.2 Ejercicio 2.2. Declaración de variables

Las declaraciones pedidas son las siguientes:

```
final int MAX_ALUMNOS = 30;  
double temperatura = 36.6;  
boolean aprobado;
```

2.3 Ejercicio 2.3. División entera

- a) $10 / 3 \rightarrow 3$. Ambos operandos son int, por lo que Java realiza división entera y trunca la parte decimal.
- b) $10.0 / 3 \rightarrow 3.3333333333333335$. Al ser uno de los operandos double, el otro se promueve a double y se realiza división en coma flotante.
- c) $10 / 3.0 \rightarrow 3.3333333333333335$. Mismo motivo que en el caso anterior: basta con que un operando sea double para que toda la expresión se evalúe como double.
- d) $10 \% 3 \rightarrow 1$. El operador % devuelve el resto de la división entera ($10 = 3 \times 3 + 1$).

2.4 Ejercicio 2.4. Promoción de tipos y casting

- a) (byte) 5 + (byte) 10 $\rightarrow$ tipo **int**, valor **15**. Aunque ambos operandos se han casteado explícitamente a byte, el operador + promueve automáticamente sus operandos byte a int antes de sumar, por lo que el resultado es siempre int.
- b) 5 + 10L $\rightarrow$ tipo **long**, valor **15L**. Al combinar un int con un long, el int se promueve a long de forma automática (ensanchamiento), sin necesidad de casting explícito.
- c) (byte) (300) $\rightarrow$ valor **44**. 300 no cabe en el rango de byte (-128 a 127), así que se trata de una conversión de estrechamiento y requiere casting explícito. Al truncar 300 a 8 bits, solo se conservan sus 8 bits menos significativos (300 en binario es 1 0010 1100; los últimos 8 bits son 0010 1100 = 44).
- d) (int) 9.99 $\rightarrow$ valor **9**. El casting de double a int es una conversión de estrechamiento que requiere (int) explícito, y trunca la parte decimal sin redondear.

2.5 Ejercicio 2.5. Evaluación en cortocircuito

Considera el fragmento de código dado. Al ejecutarlo:

```
int a = 0;
if (a != 0 && 10 / a > 1) {
    System.out.println("Cociente mayor que 1");
} else {
    System.out.println("No se cumple la condicion");
}
```

Se imprime **No se cumple la condicion**.

El operador && evalúa en cortocircuito: primero se comprueba $a \neq 0$, que es false (porque a vale 0). Como el operador && solo puede dar true si ambos operandos lo son, en cuanto el primero resulta false Java no evalúa el segundo operando ($10 / a > 1$). Esto evita precisamente una división por cero que lanzaría una `ArithmeticException` si llegara a evaluarse.

2.6 Ejercicio 2.6. Precedencia de operadores

Para boolean $r = 3 + 4 * 2 > 10 \ \&\& \ 5 - 2 == 3$;, el orden de evaluación es:

1. $4 * 2 \rightarrow 8$ (la multiplicación tiene mayor precedencia que la suma)
2. $3 + 8 \rightarrow 11$
3. $5 - 2 \rightarrow 3$ (la resta se evalúa en su propia subexpresión, con la misma prioridad relativa)
4. $11 > 10 \rightarrow \text{true}$
5. $3 == 3 \rightarrow \text{true}$
6. $\text{true} \ \&\& \ \text{true} \rightarrow \text{true}$

Por tanto, r vale **true**. El orden general es: operadores aritméticos, después relacionales ($>$, $==$), y por último el operador lógico &&.

2.7 Ejercicio 2.7. Operadores a nivel de bits

Con $a = 0b1100$ (12) y $b = 0b1010$ (10):

- a) $a \ \& \ b = 1000_2 = \mathbf{8}$
- b) $a \ | \ b = 1110_2 = \mathbf{14}$
- c) $a \ ^ \ b = 0110_2 = \mathbf{6}$
- d) $\sim a$: al invertir todos los bits de un int de 32 bits, se obtiene el complemento a dos de $a + 1$, es decir, **-13** ($\sim x$ equivale siempre $a - x - 1$).
- e) $a \ \ll \ 2 = 110000_2 = \mathbf{48}$ (desplazar 2 posiciones a la izquierda equivale a multiplicar por 4)
- f) $a \ \gg \ 1 = 0110_2 = \mathbf{6}$ (desplazar 1 posición a la derecha equivale a dividir entre 2, redondeando hacia menos infinito)

2.8 Ejercicio 2.8. El operador de asignación como expresión

Analiza el fragmento de código dado:

```
int a, b, c;
a = 5;
b = c = a + 1;
a += b -= 2;
System.out.println(a + " " + b + " " + c);
```

Se imprime **9 4 6**.

Explicación paso a paso:

1. $a = 5$; $\rightarrow$ a vale 5.
2. $b = c = a + 1$; los operadores de asignación son asociativos por la derecha, así que primero se evalúa $c = a + 1$ ($c = 6$), y el valor de esa asignación (6) se usa a su vez para $b = 6$.
3. $a += b -= 2$; de nuevo por la asociatividad por la derecha, primero se evalúa $b -= 2$, es decir, $b = b - 2 = 4$; el valor resultante de esa asignación (4) es el que se usa como operando derecho de $a += \dots$, de modo que $a = a + 4 = 9$.
4. Al final: $a = 9, b = 4, c = 6$.

Reescribiendo el fragmento con únicamente asignaciones simples, sin encadenar ni usar operadores compuestos:

```
int a, b, c;  
a = 5;  
c = a + 1;  
b = c;  
b = b - 2;  
a = a + b;  
System.out.println(a + " " + b + " " + c);
```